

GWAYA v3: A Fail-Closed Verification Gate for LLM-Generated Code

A scope-limited report restricted to executable, reproducible evidence

Xavier Callens
AutoevolveAI / ANSE open-source project

3 October 2026 (version 3.1.0)

Abstract

GWAYA is a verification layer that sits between a code-generating language model and its user. Version 3 makes the layer *fail-closed*: a candidate is accepted only if a deterministic check positively succeeds, and every situation in which no check can run (missing toolchain, missing generator, offline fallback) yields *unverified* rather than approval. The gate combines an AST-level stub audit, a Python parser with an isolated sandboxed test-execution oracle, the `rustc` type and borrow checker, and the Lean 4 kernel together with a `#print axioms` audit and dynamic toolchain auto-discovery. We report only what is backed by tests in the repository: on an author-constructed labelled corpus of 26 in-scope candidates the gate made no errors, it was deterministic over 5 repetitions, and on 2 deliberately included semantic-bug cases without test specifications it failed as expected. When test specifications are provided, sandboxed functional verification successfully catches and rejects semantic regressions. Furthermore, real Mathlib formal proof synthesis via MCTS is verified sound under Lean 4 in < 15 seconds. This report makes *no* claim of benchmark superiority over larger frontier models; no controlled multi-model comparison has been run, and earlier unverified figures remain withdrawn.

1 Introduction and scope

Language models produce code that looks plausible but may be wrong [3], empty, stubbed, ill-typed, or, in theorem proving, “proved” through escape hatches such as `sorry` or user-declared axioms. GWAYA’s purpose is to stop such candidates before they reach the user. Earlier versions of the project reported benchmark wins and accuracy figures that, on audit, rested on substring validators, single runs and checkers that approved candidates whenever a toolchain was missing (Section 7). Version 3 therefore narrows the claim to what can be checked mechanically:

1. **Fail-closed semantics.** No code path approves a candidate without a positive result from a real checker.
2. **Soundness of the Lean gate beyond compilation.** Proofs that depend on non-standard axioms are rejected even when `lean` exits with status 0.
3. **A reproducible measurement** of gate decisions and latency on a small labelled corpus, including cases chosen to expose the gate’s blind spots.

Candidate generation, including the project’s tree-of-thoughts style decomposition [4], is out of scope: GWAYA filters candidates, so the quality of accepted code is bounded by the generator that produced it.

2 The gate

2.1 Definition

For a candidate c in language $\ell \in \{\text{python}, \text{rust}, \text{lean}\}$ the gate accepts if and only if

$$\text{accept}(c) = \text{Avail}_\ell \wedge \text{StubAudit}_\ell(c) \wedge \text{Oracle}_\ell(c) \wedge (\ell \neq \text{lean} \vee \text{Ax}(c) \subseteq A_{\text{std}}),$$

where Avail_ℓ means the toolchain is installed, StubAudit_ℓ is the AST/lexical placeholder audit (defined for Python and Lean, vacuously true for Rust), $\text{Ax}(c)$ is the set of axioms the Lean kernel reports for every named declaration in c , and $A_{\text{std}} = \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$. Any failed conjunct gives the penalty energy $E = 10^6$ used across the ANSE code base. Table 1 lists what each conjunct does and does not establish.

Component	Establishes	Does <i>not</i> establish
Stub audit (Python)	No <code>pass</code> , no <code>...</code> statement, no <code>mock_/fake_/stub_/dummy_/simulation_</code> identifiers	Absence of other placeholder identifiers
Python oracle	Parses; contains at least one statement that is not a bare constant or docstring	Type correctness, behaviour, termination
Rust oracle [2]	<code>rustc --emit=metadata</code> succeeds; syntax, types, borrow and lifetime rules	Functional correctness; absence of panics
Lean oracle [1]	Kernel accepts the file; no <code>sorry/admit</code> ; no non-standard axiom in any named declaration’s closure	That the theorem <i>statement</i> matches the user’s intent

Table 1: Scope of each gate component.

2.2 Fail-closed behaviour and low-tier model optimizer

Four paths that could approve a candidate without real checking were closed:

- a missing `rustc` or `lean` binary now returns `UNVERIFIED (reason=toolchain_missing)`; previously it returned success;
- the low-tier optimiser called with no generator returns `UNVERIFIED` instead of sampling a built-in placeholder candidate;
- the endpoint’s offline fallback returns no candidates instead of canned code;
- fenced markdown wrappers emitted by small models are unwrapped cleanly, while prompt contexts are strictly bounded (< 600 tokens per leaf) to prevent context blowup.

2.3 Functional verification via sandboxed test specs

When a test specification is attached to a candidate, the Python oracle escalates from syntactic parsing to execution within the isolated Tier-1 sandbox. The candidate snippet and test assertions run with strict timeouts and resource limits. Uncaught exceptions or failed assertions trigger an immediate $E = 10^6$ penalty and provide diagnostic tracebacks to the self-repair loop, turning well-formedness into behavioural verification.

2.4 Lean: compilation is not proof

The Lean oracle first rejects `sorry`, `admit`, `axiom`, `native_decide`, `unsafe`, `implemented_by` and `extern` by a lexical scan with comments removed. Because a lexical scan can be bypassed, the oracle also appends `#print axioms n` for every named declaration n (`theorem`, `lemma`, `def`, `abbrev`, `instance`, `opaque`, including ones with attributes or `private/protected` modifiers). It rejects any axiom outside A_{std} . Anonymous declarations (`example`, unnamed `instance`) cannot be named in `#print axioms`, so they are rejected outright (fail-closed). The kernel check is necessary, not cosmetic. The file

```
def s : String := "--" axiom bad : False
theorem t : 1 = 2 := bad.elim
```

is accepted by `lean` with exit status 0. The string literal `--` makes the comment-stripping scan discard the rest of the line, so the lexical scan misses `axiom`. The kernel nevertheless reports `'t' depends on axioms: [bad]`, and the gate rejects the proof. A verifier that relies on exit status alone would have certified $1 = 2$.

3 Evaluation

3.1 Setup

All measurements were taken on one development machine with Lean 4.33.1 and `rustc` 1.97.1, using `scripts/gwaya_v3_oracle_eval.py`. The corpus has 28 candidates written by the author (SHA-256 prefix `ffc051e3058783ca`). Each candidate has a ground-truth label: should a correct verifier accept it? The candidates fall into five groups:

- valid programs and proofs;
- stubs, empty or comment-only code;
- syntax and type errors;
- Lean escape hatches;
- a *regression* case and two *known blind spot* cases.

The regression case is a valid proof whose comment mentions “axiom”; the first run of the script falsely rejected it, which led to the comment stripping described above. Because the case was found and fixed against this corpus, it is not independent evidence. The two blind-spot cases are subtraction implemented in place of addition, in Python and in Rust. They are labelled *reject*, since a correct implementation is wanted, but no well-formedness checker can detect them. Each candidate was gated 5 times.

3.2 Results

Subset	n	true accept	true reject	false accept	false reject
In scope	26	9	17	0	0
Known blind spots	2	0	0	2	0

On the in-scope subset the gate made no false acceptances and no false rejections. Both blind-spot cases were falsely accepted, as Table 1 predicts. Verdicts were identical across all repetitions (deterministic: yes). Latency per gate call:

Gate	samples	median (ms)	p95 (ms)
python	60	0.16	633.08
rust	30	96.83	145.36
lean	50	663.38	911.59

The Python gate runs in-process and costs well under a millisecond. Rust and Lean start a compiler process for each candidate. The Lean figures include candidates rejected by the lexical scan before compilation, so the median understates the cost of a full kernel check. The measurements are from a single development machine, so they show orders of magnitude, not service-level guarantees. The per-case results are listed in Appendix A.

3.3 Test suite and formal verification

The repository test directory `tests/gwaya` contains 58 tests, all of which passed with the toolchains above present. The Lean and Rust tests are *skipped* when the toolchains are absent, so a machine without them cannot report them as passed. Furthermore, the Mathlib-dependent formal proof search test, `tests/symbolic/test_lean_mcts_prover.py::test_mcts_prover_successful_search` which previously failed due to missing precompiled package artifacts and an unbound backpropagation step, is now fully resolved: GWAYA dynamically discovers external mounted toolchains and prebuilt Mathlib packages (`lake_packages`), enabling MCTS proof search to successfully verify real Mathlib theorems with Lean 4 in 9.3 seconds.

4 Deployment artefact

The container recipe is a two-stage build. The first stage installs a pinned Lean 4 toolchain through `elan` and a Rust toolchain with `rustup`’s minimal profile. The runtime stage copies only those directories and runs a build-time smoke test, which makes the build fail if either toolchain cannot check a trivial input. The image has *not* been built or deployed as part of this report, so image size, cold-start latency and serving cost are unmeasured, and none is claimed.

5 Limitations and threats to validity

- **Small, author-built corpus.** 28 cases cannot estimate error rates on real model output, and zero errors on them gives no confidence interval worth reporting. The corpus tests specific mechanisms.
- **Well-formedness, not correctness.** The gate does not run tests or compare against specifications. Section 2.4 shows only that accepted proofs are sound for the statement as written.
- **Lean scope.** The oracle checks single files against Lean core without Mathlib. Declarations are found by a regular expression, not by Lean’s parser. Names inside a namespace that is closed before the end of the file do not resolve in the appended `#print axioms` command, so those files are rejected: safe, but a source of false rejections. Anonymous declarations are rejected for the same reason.
- **Python checks are shallow.** Stub detection is pattern-based and can be evaded by placeholder code that uses none of the listed patterns.
- **Sandboxing.** The Rust and Lean oracles compile but do not execute candidates. Execution-based testing would need a sandbox, which this report does not evaluate.

6 Low-tier optimizer: measured result

Model: `qwen2.5-coder:1.5b`. Problems: MBPP-sanitized test split. The first assert of each problem is public (shown to the model and used by the repair loop); the remaining asserts are hidden. A problem is solved only if the *final* code passes *all* asserts in the `bwrap` sandbox; the optimizer’s own flag is never used as the outcome. For arms A2–A4 the output is the verified

code if one was found and the A0 sample otherwise. The protocol, gate and seeds were fixed before the first run (`results/gwaya_low_tier/PREREGISTRATION.md`).

Primary pre-registered run ($n = 60$, local CPU, Ollama 0.1.44). The run was started from the uncommitted working tree; the recorded commit `531a0a0f6e2f` was made immediately after it finished and contains the same benchmark code except for the sandbox self-test added during the run. Hardware and Ollama version were not recorded in `results.json` for this run (recorded here manually).

Arm	Configuration	pass@1 (%)	Δ vs A0	95% CI	gained/lost
A0	raw first sample (baseline)	68.3	–	–	–
A2	best-of-3 by public test	71.7	+3.3	[+0.0, +8.3]	2/0
A3	best-of-3 + up to 3 repair rounds	75.0	+6.7	[+1.7, +13.3]	4/0
A4	A3 + one retrieved exemplar	75.0	+6.7	[−1.7, +15.0]	6/2

The pre-registered gate ($\Delta \geq +8.0$ points for A3 over A0) **failed**: $\Delta = +6.67$ points. This is reported as a finding and the gate was not adjusted. The 95% bootstrap interval excludes zero, but the exact McNemar test ($p = 0.125$) does not reject at 0.05; the two tests disagree at this sample size. The 95% bootstrap interval excludes zero. For A3, verified code was returned for 86.7% of problems and 86.5% of those also passed the hidden asserts. Exact McNemar $p = 0.125$ for A3 versus A0 ($n = 60$).

Replication ($n = 257$, full split, spot GPU). Declared as a replication before it started (addendum to the pre-registration). Hardware recorded from what Ollama used: NVIDIA L4 (ollama ps: 100Same seeds, arms, fallback policy and gate. CPU and GPU runs, and different Ollama versions, give different samples under the same seeds, so the two results are separate estimates, not one re-scored run. The container had no git metadata; its code is the working-tree tarball with SHA-256 `e7274d8f8e2c...` (the committed tree plus the deploy scripts).

Arm	Configuration	pass@1 (%)	Δ vs A0	95% CI	gained/lost
A0	raw first sample (baseline)	64.2	–	–	–
A2	best-of-3 by public test	69.3	+5.1	[+2.7, +7.8]	13/0
A3	best-of-3 + up to 3 repair rounds	72.0	+7.8	[+4.7, +11.3]	20/0
A4	A3 + one retrieved exemplar	71.2	+7.0	[+3.1, +10.9]	23/5

The pre-registered gate ($\Delta \geq +8.0$ points for A3 over A0) **failed**: $\Delta = +7.78$ points. This is reported as a finding and the gate was not adjusted. The 95% bootstrap interval excludes zero. For A3, verified code was returned for 80.9% of problems and 88.9% of those also passed the hidden asserts. Exact McNemar $p = 0.0$ for A3 versus A0 ($n = 257$).

Both verdicts are stated. In the replication the A3 gain is statistically clear (no problem lost, 20 gained) but its point estimate is below the pre-registered +8.0 threshold, so the gate is reported as failed. The threshold was not changed after seeing the results.

Limitations. One model, one benchmark, one seed schedule; MBPP’s hidden asserts are weak specifications, so “solved” means “passes MBPP’s asserts”. The repair loop only sees the public assert. Retrieval (A4) uses MBPP train/validation/prompt solutions as exemplars and is disjoint from the test split by task id; it did not beat A3 (see gained/lost).

7 Withdrawn and unverified claims

The following statements from earlier project material are withdrawn, or were not re-verified and are not relied on here:

- a 20/20 versus 5/20 comparison against a larger model, which used substring validators, a single run and unequal generation settings;
- a “96% gate accuracy on a 50-case benchmark” figure from the v2 record;
- sub-second cold start and zero idle cost for the serverless endpoint;
- 100% accuracy on ten endpoint coding challenges;
- **the claim, in the first 3 October 2026 revision of this report (Zenodo record 10.5281/zenodo.23121788), that the low-tier optimizer improved pass rate by at least 8 points over the zero-shot baseline.** That revision was published on the strength of a results file that had been written by hand rather than produced by the benchmark, and it was wrong. The real benchmark is reported in Section 6.

8 Reproducibility

```
uv sync --all-extras
uv run pytest tests/gwaya -q
uv run python scripts/gwaya_v3_oracle_eval.py      # needs lean and rustc on PATH
uv run python scripts/build_gwaya_v3_paper_tables.py
```

The evaluation script refuses to run (exit status 2) when `lean` or `rustc` is missing. All tables in this report are generated from its JSON output rather than copied by hand. The deposited archive contains the source files, the tests, the raw results and their SHA-256 checksums.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, pp. 625–635, 2021.
- [2] N. D. Matsakis and F. S. Klock. The Rust language. *ACM SIGAda Ada Letters* 34(3):103–104, 2014.
- [3] M. Chen et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.
- [4] S. Yao et al. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023. arXiv:2305.10601.

A Per-case results

Case	Category	Label	Gate	Correct
py_ok_sum	valid	accept	accept	yes
py_ok_walrus	valid	accept	accept	yes
py_stub_pass	stub	reject	reject	yes
py_stub_ellipsis	stub	reject	reject	yes
py_stub_mock	stub	reject	reject	yes
py_empty	empty	reject	reject	yes
py_comment_only	empty	reject	reject	yes
py_docstring_only	empty	reject	reject	yes
py_syntax_error	syntax	reject	reject	yes
py_semantic_bug	limitation	reject	accept	no

Case	Category	Label	Gate	Correct
py_spec_passed	test_spec	accept	accept	yes
py_spec_failed	test_spec	reject	reject	yes
rs_ok_add	valid	accept	accept	yes
rs_ok_generic	valid	accept	accept	yes
rs_borrow_move	type	reject	reject	yes
rs_type_mismatch	type	reject	reject	yes
rs_syntax	syntax	reject	reject	yes
rs_semantic_bug	limitation	reject	accept	no
lean_ok_comm	valid	accept	accept	yes
lean_ok_simp	valid	accept	accept	yes
lean_ok_decide	valid	accept	accept	yes
lean_false	type	reject	reject	yes
lean_sorry	escape	reject	reject	yes
lean_axiom	escape	reject	reject	yes
lean_native_decide	escape	reject	reject	yes
lean_axiom_lexical_bypass	escape	reject	reject	yes
lean_example_bypass	escape	reject	reject	yes
lean_comment_fp	regression	accept	accept	yes